

Group 9 Assignment Report

An Evolutionary Algorithm for the Travelling Salesman Problem

Christos Zampounis Mehmet Cihan Yalcin Péter Dávid Hajdu
Pierre Kasparian Prajval Parthiban

September 30, 2026

1 Introduction

The goal of this assignment is to design, implement and experimentally evaluate an evolutionary algorithm for a combinatorial optimisation problem. We chose the Travelling Salesman Problem, whose discrete, permutation-based search space and relevance to routing and logistics make it a natural and challenging testbed for evolutionary computation. This report presents our problem formulation, the evolutionary approach and operators we developed, and an experimental analysis of its solution quality and convergence behaviour.

2 The Travelling Salesman Problem

2.1 Mathematical Formulation

The Travelling Salesman Problem (TSP) is a classical combinatorial optimisation problem that, despite its simple formulation, is difficult to solve: given a set of N cities and the distances between them, the task is to find the shortest closed tour that visits every city exactly once. The problem is NP-hard and no exact algorithm can solve it in polynomial time. Following the formulation of Potvin [1], we model the problem on a complete weighted graph $G = (V, E)$, where $V = \{1, \dots, N\}$ is the set of cities and d_{ij} is the distance between cities i and j . The decision variables are $x_{ij} \in \{0, 1\}$, where $x_{ij} = 1$ if and only if the tour travels directly from city i to city j . The objective is to minimise the total travelled distance,

$$\text{Minimise} \quad \sum_{i=1}^N \sum_{j=1}^N d_{ij} x_{ij}, \quad (1)$$

subject to the degree constraints

$$\sum_{j=1}^N x_{ij} = 1, \quad i = 1, \dots, N, \quad \sum_{i=1}^N x_{ij} = 1, \quad j = 1, \dots, N, \quad (2)$$

which require every city to have exactly one incoming and one outgoing edge, and the subtour-elimination constraints

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1, \quad \forall S \subset V, \quad 2 \leq |S| \leq N - 1, \quad (3)$$

which forbid partial cycles over any proper subset S of the cities. Without constraints (3), the model reduces to the assignment problem and its optimal solution generically consists of several disconnected subtours rather than a single tour. For symmetric instances, such as

the Euclidean TSP (ETSP) considered in this report, the optimal solution of the assignment-problem relaxation typically contains many two-vertex subtours; such instances are therefore better tackled by specialised algorithms that exploit this structure [1].

Because an evolutionary algorithm operates on orderings rather than on binary edge variables, it is convenient to state the equivalent tour-as-permutation formulation. A candidate solution is a permutation $\pi = (\pi_1, \dots, \pi_N)$ of the cities, and its cost is the length of the cycle that visits them in that order,

$$C(\pi) = \sum_{k=1}^{N-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_N, \pi_1}. \quad (4)$$

The evolutionary algorithm therefore seeks a permutation minimising Equation (4); this representation is the one adopted throughout this report.

2.2 Related Evolutionary Approaches

2.2.1 Edge-Swapping Genetic Algorithm (Liu, 2014)

Liu [2] builds a genetic algorithm around a crossover operator called *Edge Swapping* (ES), which overlays the two parent tours and partitions their edges into alternating cycles, called *M-rings*, that correspond to the regions where the parents disagree; the offspring keeps the shared edges and swaps in selected M-rings from the second parent. The search starts with a conservative *local* version of ES and switches to a *global* version once progress stalls, the latter exchanging groups of geographically close M-rings. Crucially, the population is initialised not randomly but with a greedy local search using the 2-opt neighbourhood. On benchmarks of up to 16,862 cities the algorithm reaches best-known solutions and matches. Following Liu, we therefore seed our own population with greedy nearest-neighbour tours refined by 2-opt, which gives the search a strong starting point at negligible cost.

2.2.2 Genetic-based Particle Swarm Optimization (Liao et al., 2012)

Liao et al. [3] propose GPSO, a genetic-based particle swarm optimisation for the TSP: each particle is a real-valued vector decoded into a tour by ranking its components, and crossover and mutation operators are added to the standard velocity update. We adopt the geometric construction with which they initialise the search: the geographical centre of the cities is computed, each city is assigned the angle it forms around that centre, and the cities are sorted by angle, which produces an initial non-crossing route. For large instances the authors further split the cities with Fuzzy C-Means clustering and merge the sub-tours, a decomposition we omit since the instances we consider do not exceed 100 cities. On berlin52 the authors report the near-optimal value 7544.37 in 99 of 100 runs, whereas conventional PSO found the optimum of bayg29 in only 1 of 100 runs, which motivates using this construction to start our own population with non-crossing tours instead of random permutations.

2.2.3 Genetic Algorithm with Order Crossover (Hussain et al., 2017)

Hussain et al. [4] compare crossover operators for a genetic algorithm on the TSP using the path representation, in which a chromosome is simply the order in which the cities are visited, and where classical one-point or uniform crossovers cannot be used because they create duplicate cities. They review partially mapped crossover (PMX), order crossover (OX) and cycle crossover (CX), and propose a modified cycle crossover (CX2) that recombines the parent cycles differently to obtain offspring that differ more from their parents. On twelve symmetric and asymmetric TSPLIB instances, CX2 was not universally best: it won on some instances, but PMX or OX performed similarly or better on the larger ones. We adopt OX in our own algorithm: a segment

of one parent is copied into the child, and the remaining cities are filled in the order in which they appear in the other parent, omitting those already present, so that the relative order of the parents is preserved and the child is always a valid tour.

3 Algorithm

4 Experimental part

5 Results and Analysis

6 Conclusions

References

- [1] J.-Y. Potvin, “Genetic algorithms for the traveling salesman problem,” *Annals of Operations Research*, vol. 63, no. 3, pp. 339–370, 1996. <https://doi.org/10.1007/BF02125403>.
- [2] S. Liu, “A Powerful Genetic Algorithm for Traveling Salesman Problem,” arXiv:1402.4699, 2014. <https://arxiv.org/abs/1402.4699>.
- [3] Y.-F. Liao, D.-H. Yau, and C.-L. Chen, “Evolutionary algorithm to traveling salesman problems,” *Computers & Mathematics with Applications*, vol. 64, no. 5, pp. 788–797, 2012. <https://doi.org/10.1016/j.camwa.2011.12.018>.
- [4] A. Hussain, Y. S. Muhammad, M. N. Sajid, I. Hussain, A. M. Shoukry, and S. Gani, “Genetic algorithm for traveling salesman problem with modified cycle crossover operator,” *Computational Intelligence and Neuroscience*, vol. 2017, Article ID 7430125, 7 pages, 2017. <https://doi.org/10.1155/2017/7430125>.